Compilers

Jyothi Vedurada

Why Compilers?




Microsoft
Language
Pascal
ABAP
C/AL
Cg
Rust
GDL
Bistro
JOSS
SYMPL
Cesil
Scala
Genie
Inform
SenseTalk
Blue
HAL/S
SIMSCRIPT
Revolution
Sawzall
Chapel
Rapira
CSKA
SIMPLE
ALGOL
DATATRIEVE
PL/I
ELAN
Agora
Clojure
Ajax
Not
Trac
Object
Autolt
Microcode
code
Lynx
Adenine
SAM76
Redcode
Logo
SALSA
ATS
Fjölnir
Fuxi
Assembly
TACPOL
FP
shell
E
language
A+
Lisp
VB
MicroScript
Squirrel
Windows/Dos
Kaleidoscope
IMP
C++
Fancy
SuperTalk
P
K
Visual
JavaScript
FL
JavaFX
R
Forth
b
T
Newspeak
LC-3
ABC
L
XOTcl
dc
Cayenne
Go
Delphi
DASL
D
S
Caml
Z
GameMonkey
ECMAScript
Prolog
GJ
JASS
bc
GM
Constraint
EusLisp
ObjectScript
PEARL
Oxygene
G
Programming
Java
BLISS
notation
Ruby
EXEC
RPG
M
Cyclone
CorVision
Action
REXX
F
RPL
Little
NET
Q
Simula
Boomerang
COBOL
GraphTalk
A
ChucK
Common
Agena
Strand
Turing
ISWIM
S/SL
PL/SQL
Arc
Moby
ORCA/Modula-2
Uniface
Python
J
Robot
rex
HaXe
Eiffel
SP/k
Coral
WATFOR

Why Compilers?

- Compilers are important core infrastructures for building other software.
- Compilers are everywhere!
- Machine code generation for high level languages
- Parsers for HTML in web browser
- Software testing
- Program optimization
- Malicious code detection
- Design of new computer architectures
  - Compiler-in-the-loop hardware development

Why Compilers Now?



Why Compilers Now?

- In recent years, with the rise of AI and other emerging applications, more domain specific architectures such as GPUs, TPUs or FPGAs are more prevalent.

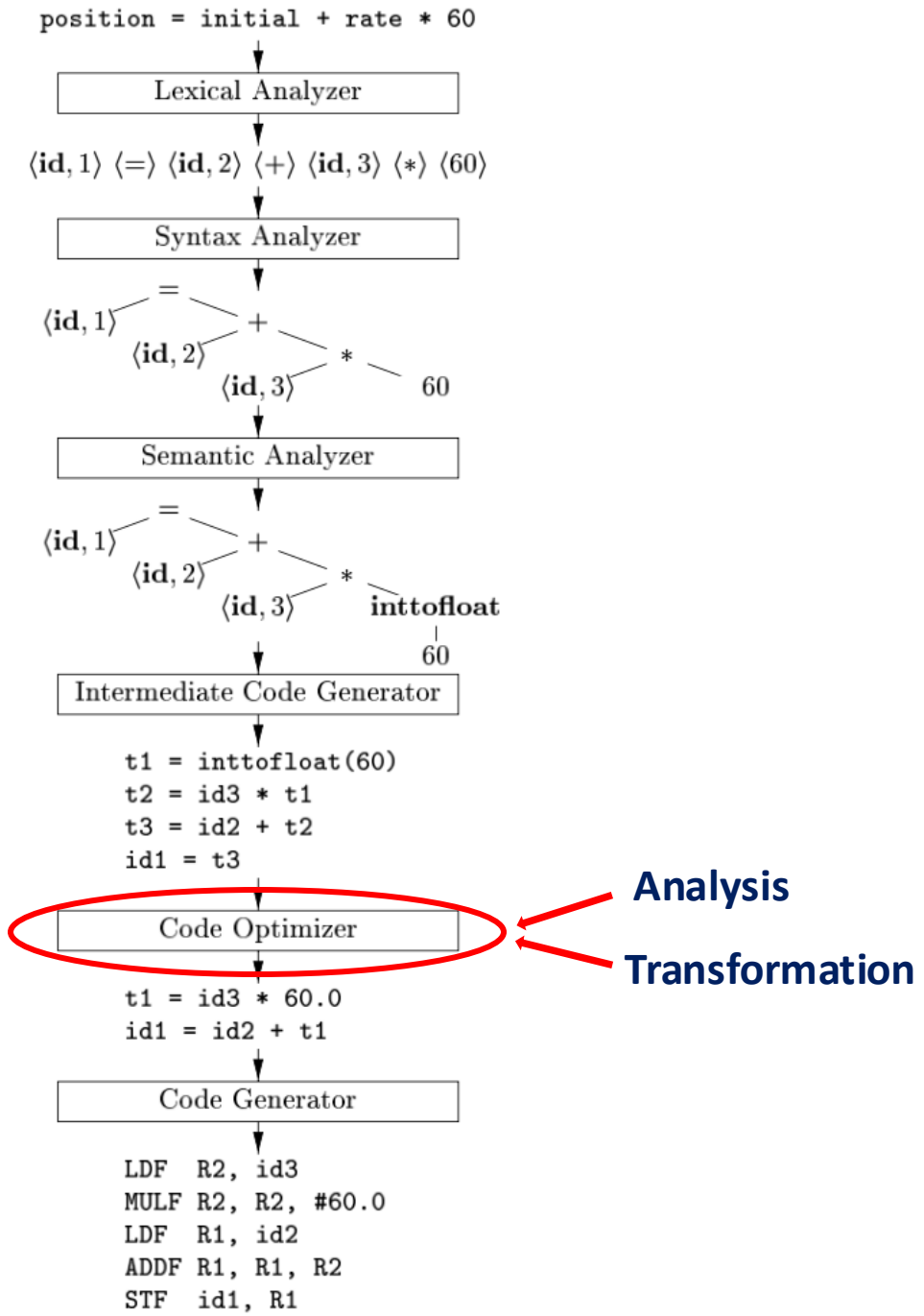
- As the complexity and diversity of applications and underlying architectures are increasing, it becomes more and more difficult to generate efficient code.

- Compilers play an increasingly important role in making programs run efficiently on the given customized platform.

Compiler Passes

1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE



Syntax Checks vs. Semantic Checks

- Lexical Checks: Checks for valid tokens by checking characters
  - E.g., Invalid identifiers: `int 9count = 5;`
  - Invalid number formats or unterminated strings: `char s = "hi;`
- Syntax Checks: Verify that tokens follow the grammar (structure) of the programming language.
  - E.g., Missing parenthesis: `if (x > 10`
- Semantic Checks: Ensure the meaning of statements is logically and contextually correct.
  - E.g., Type mismatch: `int x = "hello";`

Classwork

- Write a C program to check if every line in an input program ends with a semicolon. (Syntax check)

Complexity of Compiler Technology

- A compiler is possibly the most complex system software and writing it is a substantial exercise in software engineering
- The complexity arises from the fact that it is required to map a programmer's requirements (in a HLL program) to architectural details
- It uses algorithms and techniques from a very large number of areas in computer science
- Translates intricate theory into practice - enables tool building

Nature of Compiler Algorithms

- Draws results from mathematical logic, lattice theory, linear algebra, probability, etc.
  - type checking, static analysis, dependence analysis and loop parallelization, cache analysis, etc.
- Makes practical application of

Greedy algorithms - register allocation

Heuristic search - list scheduling
Graph algorithms - dead code elimination, register allocation
Dynamic programming - instruction selection
Optimization techniques - instruction scheduling
Finite automata - lexical analysis
Pushdown automata - parsing
Fixed point algorithms - data-flow analysis
Complex data structures - symbol tables, parse trees, data dependence graphs
Computer architecture - machine code generation

Typical Course Structure

- Surprize Quizzes: 20% (best 4/(5 to 6) exams: 5% each)
- End-Sem Exam: 25%
- Assignments: 10% (2 assign.: 5% each)
- Project: 30%
- Viva: 5%
- ~~Lab exam: 10%~~
- Class Participation: 10%



Project

- Each team has four members
- Each team will come up with a domain-specific languages
- Each team is required to write an optimizing compiler for a certain language from scratch. *Isn't it exciting?*



The beauty of Compilers

- “Theory and practice are not mutually exclusive; they are intimately connected. They live together and support each other” - Donald Knuth

- “Compiler design is a beautiful marriage of theory and practice ” - Alfred V. Aho